

BLOC 3 — CDA CESI

CESIZen

Dossier Technique — Plan de déploiement, maintenance et sécurisation

AUTEUR

Louis Beaujoin

FORMATION

CDA CESI Bloc 3

VERSION

v1.0.0 — Juillet 2026

TESTS

47 tests · CI/CD ✓

Laravel 13 · PHP 8.3 · Docker · GitHub Actions · Pest 4

Table des matières

1.	Contexte du projet	
2.	Plan de déploiement	
2.1	Environnements (DEV / TEST / PROD)	
2.2	Stack technique	
2.3	Infrastructure Docker	
2.4	Pipeline CI/CD GitHub Actions	
2.5	Sécurisation (OWASP Top 10)	
2.6	Conformité RGPD	
2.7	Procédure de rollback	
3.	Plan de maintenance	
3.1	Ticketing — GitHub Issues & Projects	
3.2	SLA et niveaux de criticité	
3.3	Veille technologique — Dependabot	
4.	Tests automatisés	
4.1	Suite Pest — 47 tests	
4.2	Tests de sécurité automatisés	
5.	Bonnes pratiques de développement	
6.	Conclusion	

CESIZen est une plateforme web de bien-être mental développée pour un Ministère fictif dans le cadre du Bloc 3 (CDA CESI). Elle propose des exercices de respiration guidés, des pages d'information sur la santé mentale, un espace personnel pour les utilisateurs connectés, et un back-office d'administration.

Objectif du Bloc 3

Déployer, maintenir et sécuriser une application web en production, en s'appuyant sur des outils professionnels (Docker, CI/CD, ticketing, tests automatisés, OWASP).

Périmètre fonctionnel

Module	Fonctionnalités	Accès
Respiration	3 exercices prédéfinis + exercice personnalisable, historique de sessions	Public (session connectée pour l'historique)
Information	Pages statiques sur la santé mentale, CRUD complet admin	Public / Admin
Authentification	Inscription, connexion, profil, mot de passe, désactivation de compte	Utilisateurs inscrits
Administration	Gestion utilisateurs, exercices, pages, activation/désactivation	Rôle Admin uniquement
API REST	Authentification par token, accès aux exercices	Token Bearer

2.1 Environnements (DEV / TEST / PROD)

Critère	DEV (WAMP)	TEST (Docker)	PROD (théorique)
Objectif	Développement quotidien	Validation intégration	Mise en service réelle
Démarrage	<code>composer dev</code>	<code>docker compose up --build</code>	Pipeline CD automatisé
Port	8000 (Artisan serve)	8080 (Apache)	443 (HTTPS Nginx)
Base de données	MySQL WAMP local	MySQL 8 container	MySQL managé (RDS)
Vite	Hot reload actif	Assets compilés	Assets compilés + CDN
APP_DEBUG	true	false	false
Ressources MIN	4 Go RAM	4 Go RAM · 3 Go disque	2 vCPU · 4 Go RAM · 50 Go SSD

2.2 Stack technique

Backend

- Laravel **13** (LTS)
- PHP **8.3**
- MySQL **8.0**
- Apache 2.4 (Docker)

Frontend / Build

- Tailwind CSS **4**
- Vite **7**
- Blade templates
- AlpineJS (respiration)

Tests & Qualité

- Pest **4** — 47 tests
- SQLite in-memory (CI)
- GitHub Actions CI/CD

Infrastructure

- Docker Compose
- GitHub Actions
- Dependabot
- SemVer v1.0.0

2.3 Infrastructure Docker

L'environnement TEST est entièrement conteneurisé via **Docker Compose**. Deux services :

Service	Image	Port exposé	Rôle
app	Build local (PHP 8.3 Apache + Node 20)	8080→80	Application Laravel + Apache
db	mysql:8.0	Non exposé	Base de données persistante

Script de démarrage — entrypoint.sh

Le script `docker/entrypoint.sh` gère le démarrage séquentiel :

```
#!/bin/bash
set -e

# Crée .env depuis .env.example si absent (le vrai .env n'est jamais dans l'image)
[ ! -f .env ] && cp .env.example .env

# Génère une clé applicative si elle n'est pas fournie en variable d'environnement
[ -z "$APP_KEY" ] && php artisan key:generate --force

# Attend que MySQL soit prêt, puis exécute les migrations
until php artisan migrate --force 2>&1; do
    echo "Base de données non disponible, nouvelle tentative dans 3s..."
    sleep 3
done

# Insère les données de démo si demandé (DB_SEED=true)
if [ "${DB_SEED:-false}" = "true" ]; then
    php artisan db:seed --force
fi

exec apache2-foreground
```

Pourquoi copier `.env.example` ?

Le fichier `.env` réel est exclu de l'image Docker via `.dockerignore` (bonne pratique de sécurité — il contient les secrets). Le script en crée une copie depuis `.env.example` au premier démarrage, avant que `php artisan key:generate` ne soit appelé.

Commandes Docker

```
docker compose up -d --build    # Build + démarrage (port 8080)
docker compose exec app php artisan db:seed --force  # Données de démo
docker compose down             # Arrêt propre
docker compose down -v          # Suppression volume (remise à zéro)
```

Données de démo disponibles sur <http://localhost:8080>

2.4 Pipeline CI/CD — GitHub Actions

Le pipeline est défini dans `.github/workflows/ci.yml` et comprend **2 jobs séquentiels** :

Job	Déclencheur	Étapes	Critère go/no-go
tests	Push sur main ou develop	PHP 8.3 · Composer · SQLite · Pest	47/47 tests passent
docker	<code>needs: tests</code> uniquement	Build de l'image Docker	Build sans erreur

Blocage automatique

Si un seul test échoue dans le job `tests`, le job `docker` ne démarre pas. Le code cassé ne peut pas atteindre l'environnement de test.

Extrait du workflow

```
jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup PHP 8.3
        uses: shivammathur/setup-php@v2
        with: { php-version: '8.3' }
      - run: composer install --no-interaction
      - run: cp .env.example .env && php artisan key:generate
      - run: php artisan test --no-coverage # 47 tests

  docker:
    needs: tests # ← bloquant
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t cesizen-app .
```

2.5 Sécurisation — OWASP Top 10

Risque OWASP	Implémentation dans CESIZen	Statut
A01 — Contrôle d'accès	Middleware <code>auth</code> sur toutes les routes protégées, rôles <code>admin / user</code>	✓ Couvert
A02 — Cryptographie	Bcrypt 12 rounds (<code>cast: 'hashed'</code>), HTTPS en prod, secrets dans <code>.env</code>	✓ Couvert
A03 — Injection	Eloquent ORM (requêtes préparées), validation Laravel sur tous les formulaires	✓ Couvert
A05 — Mauvaise config	<code>APP_DEBUG=false</code> en prod/test, <code>.env</code> exclu de git	✓ Couvert
A07 — Auth défaillante	Throttle <code>5 req/min</code> sur <code>/connexion</code> et <code>/api/login</code> , sessions sécurisées	✓ Couvert
A03/A07 — XSS	Blade <code>{{ }}</code> (échappement auto), cast <code>(int)</code> sur paramètres JS injectés	✓ Couvert
A01 — CSRF	Token <code>@csrf</code> sur tous les formulaires POST	✓ Couvert

SecurityHeadersMiddleware

Middleware Laravel ajoutant 4 en-têtes HTTP de sécurité sur toutes les réponses web :

```
// app/Http/Middleware/SecurityHeadersMiddleware.php
$response->headers->set('X-Content-Type-Options', 'nosniff');
$response->headers->set('X-Frame-Options', 'SAMEORIGIN');
$response->headers->set('Referrer-Policy', 'strict-origin-when-cross-origin');
$response->headers->set('Permissions-Policy', 'geolocation=(), microphone=(), camera=()');
```

Protection brute-force (throttle)

```
// routes/web.php
Route::post('/connexion', [AuthController::class, 'login'])->middleware('throttle:5,1');

// routes/api.php
Route::post('/login', [AuthApiController::class, 'login'])->middleware('throttle:5,1');
```

Après 5 tentatives par minute, la réponse est `HTTP 429 Too Many Requests`. Vérifié en test automatisé ET sur l'application live Docker.

Protection XSS — Paramètres JavaScript

```
// app/Http/Controllers/BreathingController.php
$inspiration = (int) $request->get('inspiration', 4);
$apnea       = (int) $request->get('apnea', 7);
$expiration  = (int) $request->get('expiration', 8);
$cycles      = (int) $request->get('cycles', 4);
```

Le cast `(int)` neutralise toute injection (ex : `5;alert(1)//` → `5`).

2.6 Conformité RGPD

Principe RGPD	Mise en œuvre
Minimisation des données	Seuls nom, email, mot de passe hashé collectés à l'inscription. Aucune donnée de tracking.
Durée de conservation	Comptes inactifs > 3 ans supprimés. Sessions expirées > 90 jours purgées.
Droit d'accès	Profil utilisateur consultable à tout moment dans l'espace personnel.
Droit de rectification	Modification email, nom et mot de passe depuis le profil.
Droit à l'effacement	Suppression de compte possible (admin) — supprime toutes les données associées.
Sécurité des données	Bcrypt 12 rounds, HTTPS en prod, pas de données sensibles dans les logs.
Notification CNIL	En cas de violation : notification CNIL sous 72h, utilisateurs concernés sous 72h.

Limitation de ce projet

Ce projet étant éducatif, il n'y a pas de DPO désigné ni de politique de confidentialité publiée. En production réelle, ces éléments seraient obligatoires (Art. 37 et 13 RGPD).

2.7 Procédure de rollback

En cas de régression critique après une mise en production, deux niveaux de rollback sont disponibles :

Niveau 1 — Rollback du code (Git)

```
# Option A : Annuler un commit spécifique (préféré – préserve l'historique)
git revert <hash-du-commit-problématique>
git push origin main

# Option B : Revenir à un tag stable
git checkout v1.0.0
git push origin main --force    # Uniquement si aucune autre personne ne travaille sur la branche
```

Niveau 2 — Rollback de la base de données

```
# Annuler la dernière migration
php artisan migrate:rollback

# Restaurer depuis une sauvegarde mysqldump
mysql -u root -p cesizen_prod < backup_avant_migration.sql
```

Critères de déclenchement

Scénario	Délai de déclenchement	Action
Erreur 500 en production	Immédiat (< 5 min)	Rollback code + redémarrage
Régression fonctionnelle majeure	< 30 min (après constat)	Rollback code
Corruption de données	Immédiat	Rollback BDD depuis backup
Vulnérabilité sécurité critique	Immédiat (maintenance)	Rollback + patch prioritaire

Bonne pratique

Toujours effectuer un `mysqldump` de la base de données AVANT une migration en production. Les tags Git (`git tag v1.0.0`) permettent de revenir à un état stable connu en quelques secondes.

3.1 Ticketing — GitHub Issues & Projects

Le suivi est centralisé dans **GitHub Issues** avec un board **GitHub Projects** en vue Kanban :

Colonne	Signification
Backlog	Tickets créés, non planifiés
En cours	Développement actif
En revue	PR ouverte, en attente de merge
Terminé	Mergé sur main, fermé

Labels de criticité

Label	Couleur	Description
bloquant-critique	Rouge	Service indisponible ou perte de données
bloquant-majeur	Orange	Fonctionnalité principale dégradée
mineur	Bleu	Gêne sans impact métier
amélioration	Gris	Nouvelle fonctionnalité ou UX

Des **templates GitHub** pré-remplis

guident la saisie des tickets : `bug_report.md` (étapes de reproduction, environnement, criticité) et `feature_request.md` (valeur métier, cas d'usage, critères d'acceptation).

3.2 SLA et niveaux de criticité

Criticité	Prise en compte	Correction	Exemple
Bloquant critique	< 1h	< 3h ouvrées	Erreur 500 sur la page de connexion
Bloquant majeur	< 4h	< 24h ouvrées	Exercice de respiration ne se lance pas
Mineur	< 48h	Prochain sprint	Faute d'orthographe dans une page info

3.3 Veille technologique — Dependabot

Le fichier `.github/dependabot.yml` configure la mise à jour automatique des dépendances :

```
version: 2
updates:
  - package-ecosystem: "composer"    # PHP / Laravel
    schedule: { interval: "weekly" }
  - package-ecosystem: "npm"          # JS / Tailwind / Vite
    schedule: { interval: "weekly" }
  - package-ecosystem: "github-actions"
    schedule: { interval: "monthly" }
```

Chaque mise à jour génère une Pull Request automatique que je peux valider après avoir vérifié le CHANGELOG et que les 47 tests passent toujours.

4.1 Suite Pest — 47 tests · 118 assertions

Résultat final — 3 juillet 2026

Tests : **47 passés** (0 échoué) · Assertions : **118** · Durée : **2.51s** · CI GitHub Actions :  success

Suite	Fichier	Tests	Couverture
Unit / Modèles	BreathingExerciseModelTest	3	getCycleDuration, scope active, relation
Unit / Modèles	UserModelTest	4	isAdmin, hash password, fillable
Unit / Modèles	InformationPageModelTest	2	scope published, scope ordered
Feature / Auth	AuthTest	13	Inscription, connexion, profil, admin, permissions
Feature / Respiration	BreathingTest	12	CRUD exercices, sessions, paramètres custom
Feature / Information	InformationTest	6	CRUD pages, accès public/admin
Feature / Sécurité	SecurityTest	3	En-têtes HTTP, throttle 429, protection XSS
Unit + Feature	ExampleTest × 2	2	Sanity checks
TOTAL		47	118 assertions

Configuration Pest

```
# composer.json
"scripts": {
    "test": "php artisan test --no-coverage"
}

# Lancer les tests
composer test    # → 47 tests, ~2.5s, SQLite in-memory
```

4.2 Tests de sécurité automatisés

Le fichier `tests/Feature/SecurityTest.php` vérifie automatiquement les 3 mécanismes de sécurité :

```
<?php
use Illuminate\Foundation\Testing\RefreshDatabase;
```

```

uses(RefreshDatabase::class); // Migrations auto en SQLite in-memory

// Test 1 – En-têtes de sécurité sur toutes les pages
test('security headers are present on home page', function () {
    $response = $this->get('/');
    $response->assertHeader('X-Content-Type-Options', 'nosniff');
    $response->assertHeader('X-Frame-Options', 'SAMEORIGIN');
    $response->assertHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
    $response->assertHeader('Permissions-Policy', 'geolocation=(), microphone=(), camera=()');
});

// Test 2 – Throttle brute-force : 429 après 5 tentatives
test('throttle middleware blocks after 5 login attempts', function () {
    for ($i = 0; $i < 5; $i++) {
        $this->post('/connexion', ['email' => 'brute@test.com', 'password' => 'wrong']);
    }
    $response = $this->post('/connexion', ['email' => 'brute@test.com', 'password' => 'wrong']);
    $response->assertStatus(429);
});

// Test 3 – Protection XSS : cast (int) neutralise l'injection
test('xss cast int protection on breathing page', function () {
    $response = $this->get('/respiration?inspiration=5%3Balert(1)%2F%2F');
    $response->assertStatus(200);
    $response->assertDontSee('alert(1)', false);
});

```

Pratique	Outil / Convention	Exemple
Versioning sémantique	SemVer (MAJEUR.MINEUR.PATCH)	v1.0.0 — première version stable
Messages de commit	Conventional Commits	feat: fix: docs: test: chore:
Branches	Git Flow simplifié	main / develop / feature/*
Code review	Pull Requests GitHub	Checklist tests + sécurité + revue
README	Documentation in-repo	Installation DEV + Docker + tests + sécurité
Secrets	.gitignore + .env.example	.env jamais commité, APP_DEBUG=false
Dépendances	Dependabot hebdomadaire	PRs automatiques + vérification CI
Conteneurisation	Docker + .dockerignore	Image reproductible, secrets hors image

Architecture des branches

```
main      ← Code stable, livraisons en production (merge via PR uniquement)
├── develop ← Intégration continue des développements
│   └── feature/* ← Fonctionnalité isolée, créée depuis develop
```

Flux : feature/* → PR → develop → PR → main
Tag SemVer apposé sur chaque merge dans main : v1.0.0, v1.1.0, etc.

CESIZen constitue un exemple concret de projet web professionnel intégrant l'ensemble des compétences du Bloc 3 :

Déploiement

- 3 environnements distincts (DEV/TEST/PROD)
- Docker Compose reproductible
- Pipeline CI/CD GitHub Actions
- Rollback documenté et testé

Sécurité

- 7 risques OWASP Top 10 couverts
- En-têtes HTTP de sécurité
- Throttle brute-force
- Tests de sécurité automatisés

Maintenance

- Ticketing GitHub Issues + Projects
- SLA formalisé par criticité
- Dependabot actif
- Templates bug/feature

Qualité

- 47 tests automatisés (118 assertions)
- Conventional Commits + SemVer
- Git Flow structuré
- README complet

État final du projet — 3 juillet 2026

- 47/47 tests passent en CI (GitHub Actions )
- Application accessible sur Docker à <http://localhost:8080>
- Données de démo disponibles (admin@cesizen.fr / password)
- Code disponible sur GitHub : github.com/louisbeaujoin/cesizen